

The background features a grayscale profile of a man's head facing left. Overlaid on his hair is a circular pattern of handwritten-style code, including 'uz', 'uz.', and 'uzn'. Large, semi-transparent Japanese text '本当の私は' (My real self) is visible behind the main title, and 'はじまる!!' (It begins!!) is at the bottom.

ウズウズカレツジ プログラマーコース

APIと外部クラスの利用



java開発陣

▼自作のソースコード

```
class Sample1_15_1 {  
^   public static void main(String[] args) {  
^   ^   ^  
^   ^   int x = Integer.parseInt( args[0] );  
^   ^   ^  
^   ^   System.out.println( x );  
^   ^   ^  
^   }  
^ }
```

parseInt
メソッド

println
メソッド

API

java開発陣が用意してくれている
便利なソースコード群

必要なときに
必要な機能だけ
呼び出すことが可能

《APIとは》

□API (Application Program Interface)とはJavaが提供してくれている便利なソースコード群のことです。

これまで当たり前のように使ってきたprintlnメソッドやparseIntメソッドもこのAPIで定義されているものになります。

APIを利用することで自分自身でプログラムを書かずとも、高度で複雑なプログラムを実装することが可能になります。

(printlnメソッドも実は高度なプログラムが組みられています)

□APIにどのようなクラスおよびメソッドが存在するか、またそれらの使用方法については、Javaの提供元であるORACLEのWebページ (APIドキュメント) から確認することが可能です。

<https://docs.oracle.com/javase/jp/8/docs/api/>

～APIの格納場所～

JDK(Java開発環境)

- Javaの開発で必要になる機能群

javac

コンパイラ

jar

Jarファイル
関連機能

javadoc

仕様書の
自動生成

...

JRE(Java実行環境)

- Javaの実行で必要になる機能群

JVM

バイトコードファイルの
内容にした以外処理実行
する司令塔

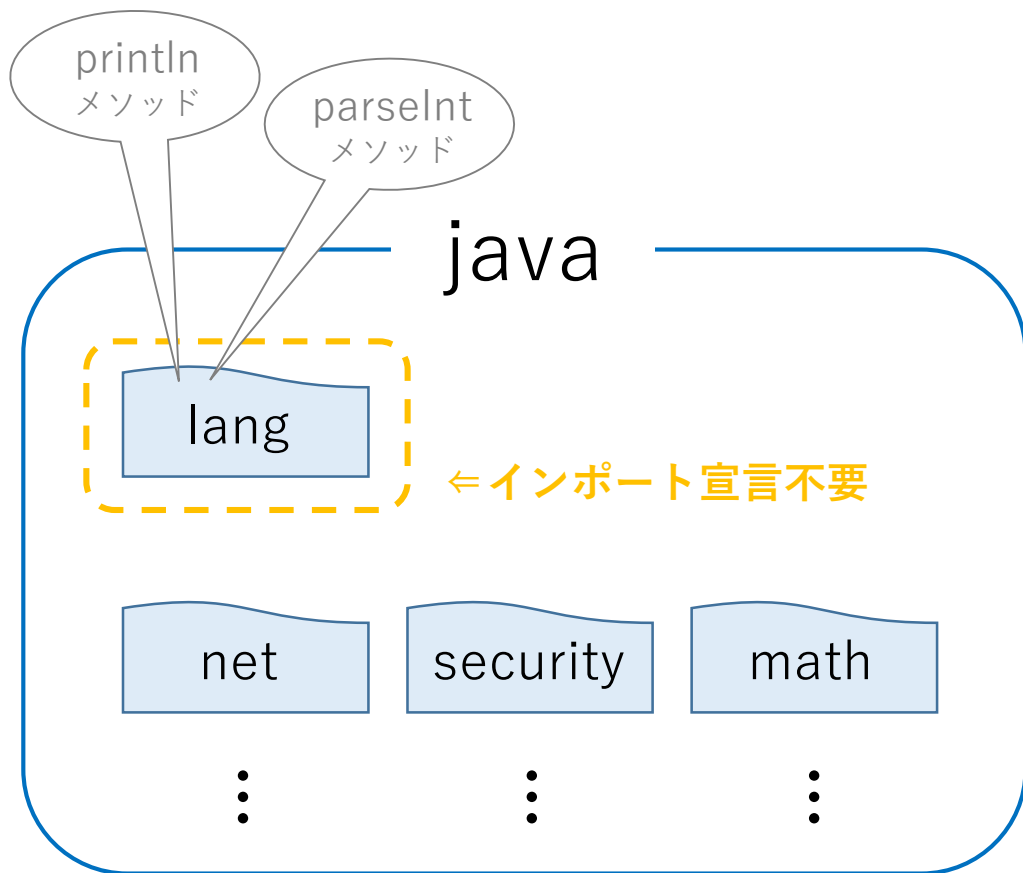
API

Java SE, Java EEなど
プログラムを実行するうえで
使用される便利な機能群

以下で管理されています

『JDKのフォルダ > jre > lib > rt.jar』

※rt.jarは圧縮ファイルなので確認には解凍が必要です。
(オススメの解凍ツール：7-Zip)



《インポート》

□APIは**パッケージ**と呼ばれるフォルダのようなもので管理されています。

まずすべてのAPIは`java`という名前のパッケージの中に入っており、その下で用途別のパッケージに分けられて管理されています。

例えば`java.security`パッケージにはセキュリティに関する機能を持ったクラス群が、`java.net`パッケージにはネットワークアプリケーションに関する機能を持ったクラス群が格納されています。
(「`.`」には「～の」という意味があります。)

□APIは基本的に使用したいクラスを**インポート**することで使用可能になります。インポートの方法は以下の通りです。

(クラスブロックの外側で) **import** パッケージ名.クラス名

□APIの中でも**`java.lang`パッケージ**で管理されているクラス群は、その利用頻度があまりに高いため**インポート不要で使用可能**となっております。これまで使用してきた`println`メソッド、`parseInt`メソッド、`String`メソッドはすべて`java.lang`パッケージに存在していたため当たり前のように使用することが可能となっております。

～インポートのイメージ～

《インポート》

```
import kitchen.microwave ;
```

kitchenパッケージ



インポートして
メモリ上に置くことで
使える状態になる

ソース
コード

API

～APIの活用～

インポート

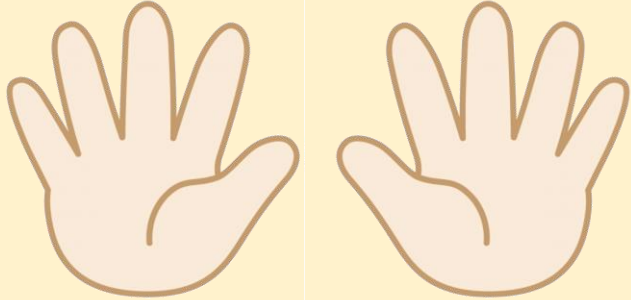
java.mathパッケージのBigDecimalクラスを使える状態にする。
(java.langパッケージに含まれるクラスは不要)

API (BigDecimalクラス) の活用

```
import java.math.BigDecimal;
class Sample1_15_2{
    public static void main (String[] args) {
        //APIの活用 (BigDecimalによる小数計算)
        System.out.println("▼ 『1.5 - 6 * 0.1』 の計算 (double型) ");
        System.out.println( 1.5 - 6 * 0.1 );
        System.out.println("▼ 『1.5 - 6 * 0.1』 の計算 (BigDecimal型) ");
        BigDecimal b1 = new BigDecimal( 1.5 );
        BigDecimal b2 = new BigDecimal( -6 );
        BigDecimal b3 = new BigDecimal( "0.1" );
        BigDecimal result = b1.add( b2.multiply( b3 ) );
        System.out.println(result);
    }
}
```

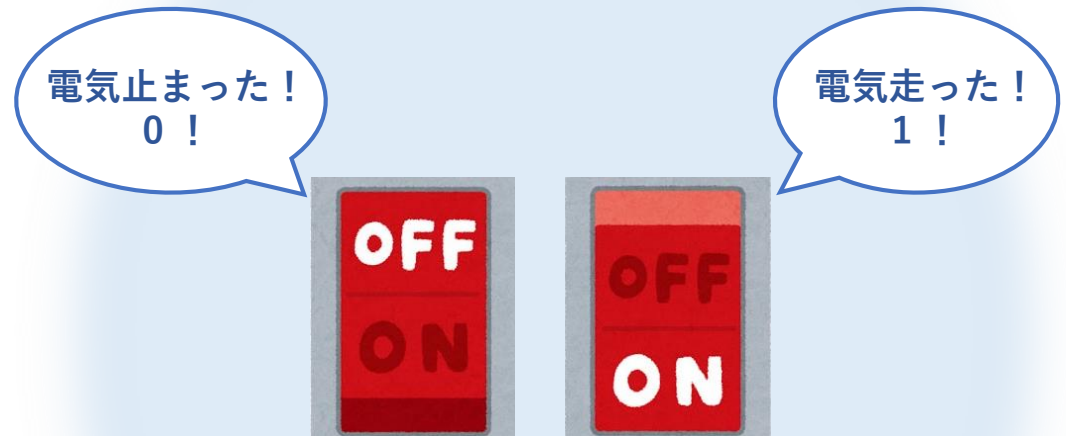
～データの実際の姿～

人間は 10進数の世界



0～9の10種類の数字で1桁を表現し、
その組み合わせで全ての数値を表現する

コンピューターは 2進数（0と1）の世界



0,1の2種類の数字で1桁を表現し、
その組み合わせで全ての情報を表現する

～データの実際の姿～

人間は
10進数の世界

10進数：桁上がりのルールが 10^n

10^2 10^1 10^0
100 10 1

1 2 3

$$100 \times 1 + 10 \times 2 + 1 \times 3$$

コンピューターは
2進数（0と1）の世界

2進数：桁上がりのルールが 2^n

2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0
128 64 32 16 8 4 2 1

0 1 1 1 1 0 1 1

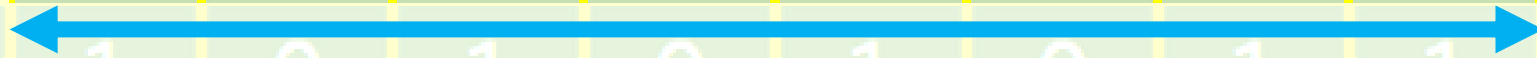
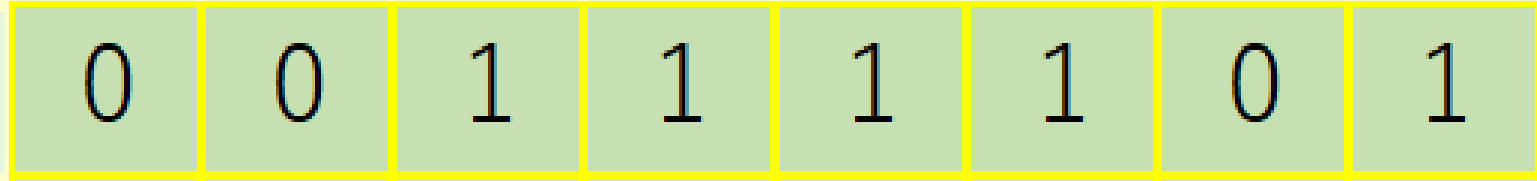
$$128 \times 0 + 64 \times 1 + 32 \times 1 + 16 \times 1 + 8 \times 1 + 4 \times 0 + 2 \times 1 + 1 \times 1$$

=

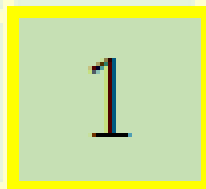
～データの実際の姿～

メモリの 拡大イメージ

～データの実際の姿～



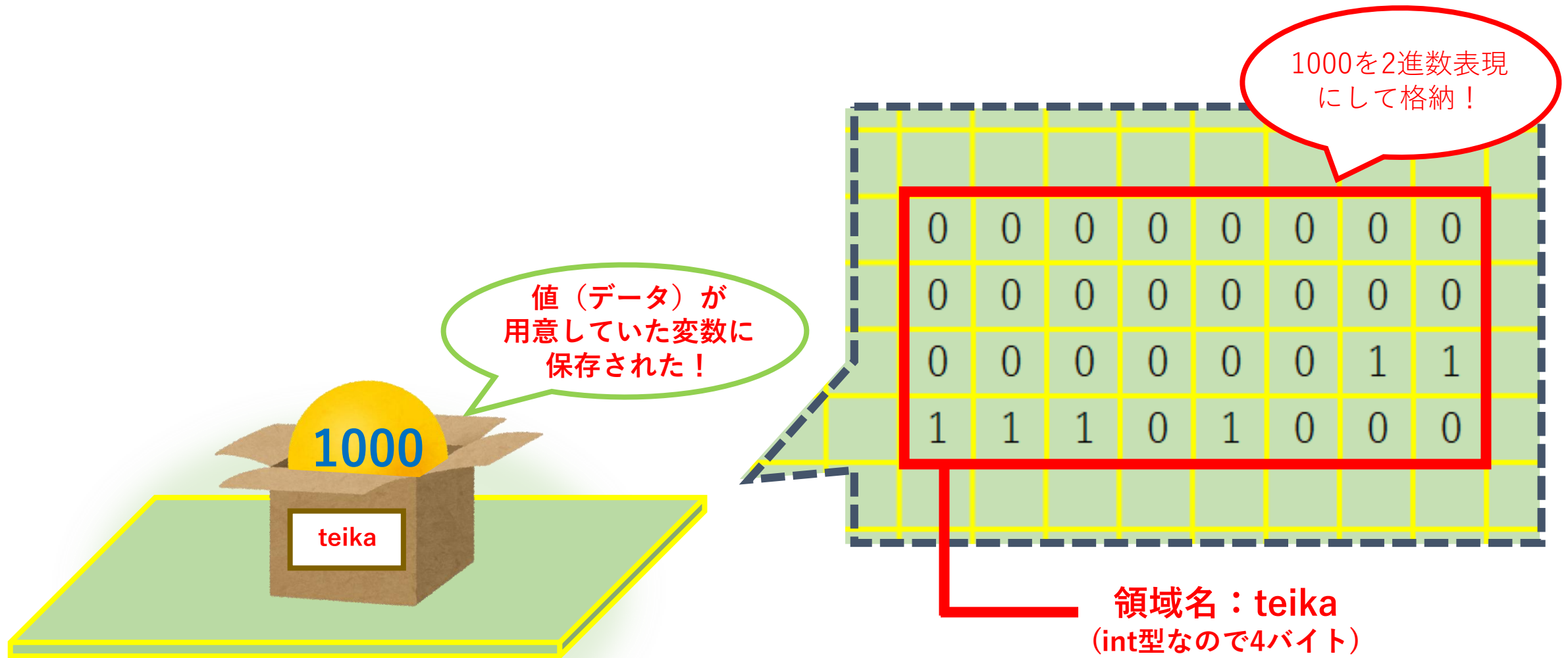
1バイト = 8ビット



1ビット

0か1のどちらか
だけ保存が可能

～データの実際の姿～



～データの実際の姿～

人間は 10進数の世界

10進数：桁上がりのルールが 10^n

$$\begin{array}{cccc} 10^0 & 10^{-1} & 10^{-2} & 10^{-3} \\ 1 & 0.1 & 0.01 & 0.001 \end{array}$$

$$10^{-x} = \frac{1}{10^x}$$

1 . 6 2 5

$$1 \times 1 + 0.1 \times 6 + 0.01 \times 2 + 0.001 \times 5$$

=

コンピューターは 2進数（0と1）の世界

2進数：桁上がりのルールが 2^n

$$\begin{array}{cccc} 2^0 & 2^{-1} & 2^{-2} & 2^{-3} \\ 1 & 0.5 & 0.25 & 0.125 \end{array}$$

$$2^{-x} = \frac{1}{2^x}$$

1 . 1 0 1

$$1 \times 1 + 0.5 \times 1 + 0.25 \times 0 + 0.125 \times 1$$

～小数データはITの世界ではズレる！～

人間は
10進数の世界

10進数：桁上がりのルールが 10^n

10^0 10^{-1}

1 0.1

1.2

X
=

コンピューターは
2進数（0と1）の世界

2進数：桁上がりのルールが 2^n

2^0 2^{-1} 2^{-2} 2^{-3} 2^{-4} 2^{-5} 2^{-6}

1 0.5 0.25 0.125 0.0625 0.03125 0.015625

1.001100...

10進数の「1.2」は
2進数では表し切れない！

～小数データはITの世界ではズレる！～

振り込んだ額と違う・・・！？



発注数がおかしいぞ・・・！？



厳密な小数の計算をする際には
特殊な処理が必要になる



BigDecimalの活用

～BigDecimalクラスの活用～

```
import java.math.BigDecimal;
class Sample1_15_2{
    public static void main (String[] args) {
        //APIの活用 (BigDecimalによる小数計算)
        System.out.println("▼ 『1.5 - 6 * 0.1』 の計算 (double型) ");
        System.out.println( 1.5 - 6 * 0.1 );
        System.out.println("▼ 『1.5 - 6 * 0.1』 の計算 (BigDecimal型) ");
        BigDecimal b1 = new BigDecimal( 1.5 );
        BigDecimal b2 = new BigDecimal( -6 );
        BigDecimal b3 = new BigDecimal( "0.1" );
        BigDecimal result = b1.add( b2.multiply( b3 ) );
        System.out.println(result);
    }
}
```

```
C:\¥Workspace>java Sample1_15_2
▼ 『1.5 - 6 * 0.1』 の計算 (double型)
0.8999999999999999
▼ 『1.5 - 6 * 0.1』 の計算 (BigDecimal型)
0.9
```

～BigDecimalクラスの活用～

メモリ上
(2進数の世界)

10進数の世界

1.5

b1

(BigDecimal型)

10進数の世界

-6

b2

(BigDecimal型)

10進数の世界

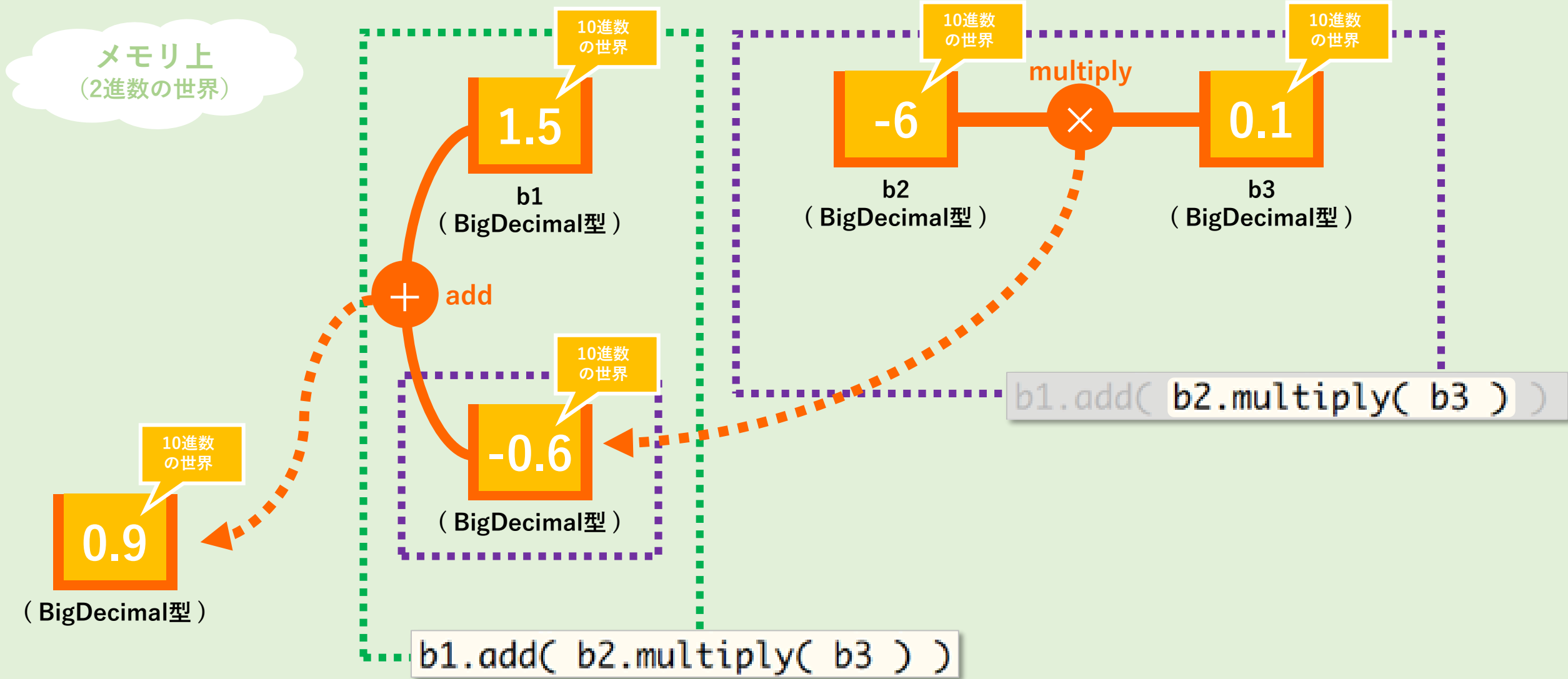
0.1

b3

(BigDecimal型)

```
BigDecimal b1    = new BigDecimal( 1.5 ) ;  
BigDecimal b2    = new BigDecimal( -6  ) ;  
BigDecimal b3    = new BigDecimal( "0.1" ) ;
```


～BigDecimalクラスの活用～

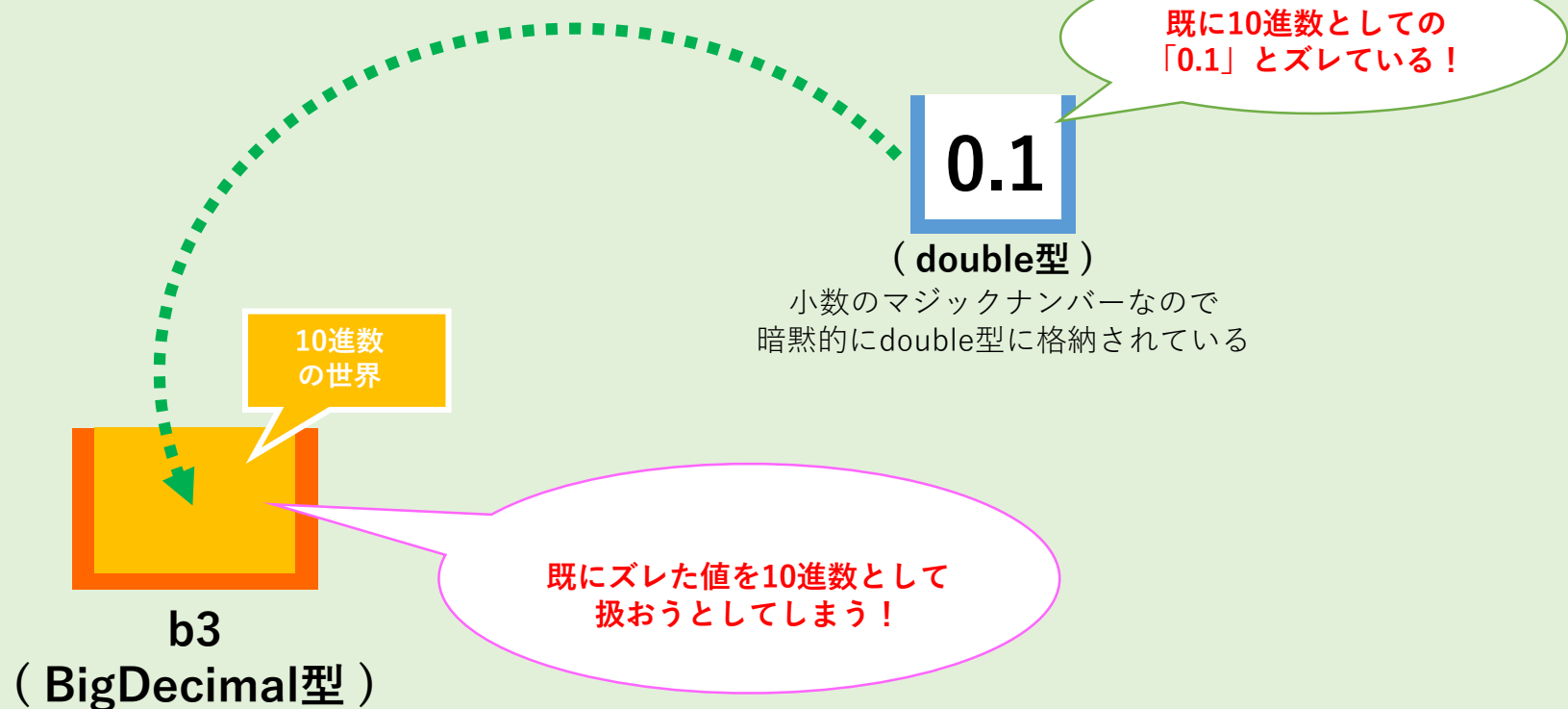


～BigDecimalクラスの活用～

メモリ上
(2進数の世界)

↓ もしも引数が「0.1」だったら・・・

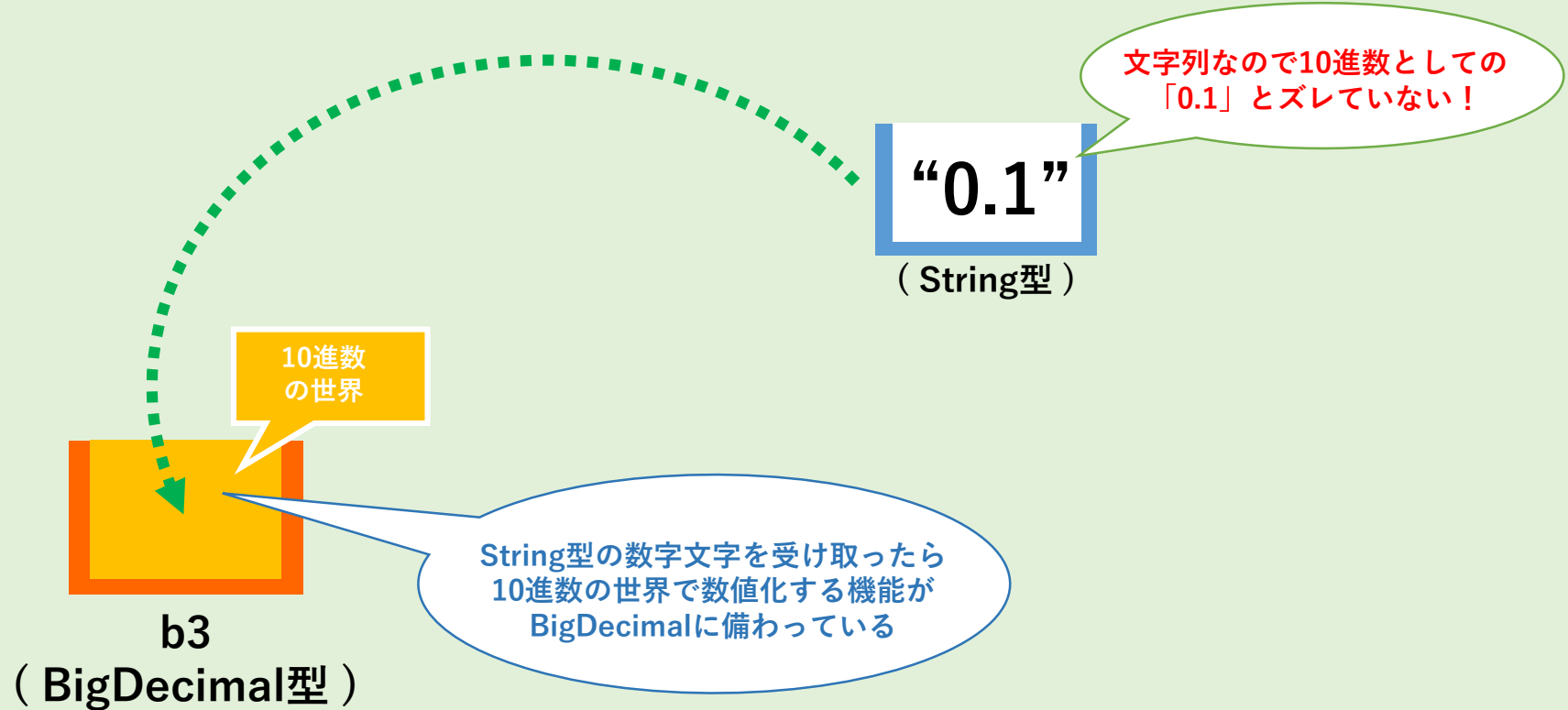
```
BigDecimal b3 = new BigDecimal( 0.1 );
```

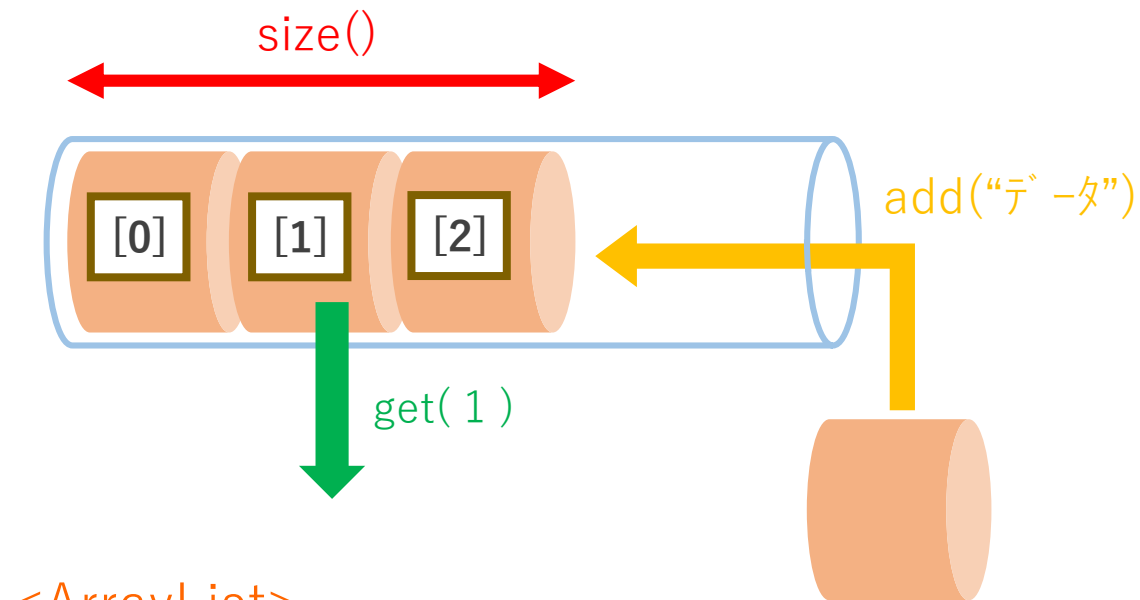


～BigDecimalクラスの活用～

メモリ上
(2進数の世界)

```
BigDecimal b3 = new BigDecimal( "0.1" );
```

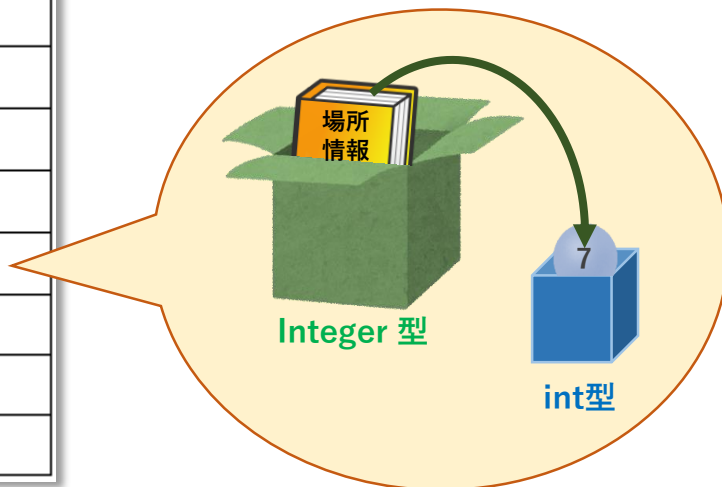




<ArrayList>

対応するクラス	基本型
Boolean	boolean
Character	char
Byte	byte
Short	short
Integer	int
Long	long
Float	float
Doubule	double

<ラッパークラス>



≪ ArrayList ≫

- ArrayListは要素数を自由に変更できる配列のようなものです。
以下の方法で生成します。

`List<型> 変数名 = new ArrayList<型>();`

- ArrayListで提供されている主な機能は以下です。
 - get(インデックス)**：インデックスに該当するデータを参照
 - add(データ)**：ArrayListにデータを追加します。追加された順に0からインデックスが振られます。
 - size()**：ArrayListの要素数を求めます。
 - isEmpty()**：リストに要素がない場合trueを返します。
 - remove(インデックス)**：インデックスに該当するデータを削除

- ArrayListはjava.utilパッケージで管理されているAPIであるため、
使用時はインポートする必要があります。
インポートの際は『java.util.ArrayList』だけでなく
『**java.util.List**』もインポートしなければならない点に気を付けて
ください。

- 配列同様に同じ型の要素を扱いますが、**参照型しか扱えない**という点
でも大きく違います。プリミティブ型のデータを扱いたい場合は、
左表のような**対応するラッパークラス**を型として指定しなければ
なりません。

<演習：Ex1 15 1>

[illegible]

<演習：Ex1_15_2>

```
/*-< 演習：Ex1_15_2 >-----<
コメントに従って2次元配列からデータを抽出して画面に表示するプログラムを作ってください。<
-----*/<

class Ex1_15_2{<
^   public static void main (String[] args) {<
^   ^   <
^   ^   /*<
^   ^   **犬の情報を持つ配列。格納されている情報は以下のとおり。<
^   ^   ** - dogs[x][0]：名前<
^   ^   ** - dogs[x][1]：年齢<
^   ^   ** - dogs[x][2]：性別<
^   ^   */<
^   ^   String[][] dogs = {<
^   ^       {"モコ" , "4歳" , "メス"} ,<
^   ^       {"ムギ" , "2歳" , "オス"} ,<
^   ^       {"ブブ" , "3歳" , "メス"} ,<
^   ^       {"シロ" , "8歳" , "オス"} ,<
^   ^       {"ブブ" , "3歳" , "メス"} ,<
^   ^       {"ペロ" , "1歳" , "メス"} ,<
^   ^       {"ルル" , "14歳" , "オス"} ,<
^   ^       {"モモ" , "10歳" , "メス"} ,<
^   ^       {"ポチ" , "6歳" , "オス"} ,<
^   ^       {"エル" , "2歳" , "メス"} <
^   ^   };<
^   ^   <
^   ^   /*以下のプログラムを作成してください。<
^   ^   ** ①String型を扱うArrayList「adultFemaleDogsList」を作成する<
^   ^   ** ②2次元配列dogsから以下の条件にあてはまる名前を抽出し、すべてadultFemaleDogsListに格納する<
^   ^   **     - 4歳以上<
^   ^   **     - メス<
^   ^   **     ※ヒント：年齢の文字列から「歳」を除去する方法について調べてみよう！<
^   ^   ** ③adultFemaleDogsListに格納されているすべてのデータをprintlnで表示する<
^   ^   */<
^   ^   <
^   ^   <
^   }<
^ }<
```